AWS CLOUD COMPUTING · PROJECT 26

Cloud-Based Image Storage and Gallery System



A stateless web application on Amazon EC2 that keeps every uploaded image as an object in Amazon S3.

Name · Roll number · Class · Date

CONTENTS

What this presentation covers

01 The problem

Why one server is the wrong place for files

02 Objective and scope

What the system does, and what it does not

03 The AWS services

EC2, S3 and the role that connects them

04 Architecture

How a request travels from browser to bucket

05 Security

Keeping the bucket private with signed links

06 Demonstration

The running application and its test results

THE PROBLEM

Why not just store images on the server?

Build this the ordinary way and every photo lands on one machine's disk. Four things then go wrong, and all of them go wrong quietly.

Disk fills up

Storage is fixed. Photos pile up until uploads start failing.

One copy only

If the machine dies, every uploaded image dies with it.

Cannot scale out

A second server holds different files, so users see different galleries.

Manual backups

Someone has to remember to copy the folder somewhere safe.

The cloud answer is a split: the application stays on EC2, the files move to S3.

OBJECTIVE

What the system does

Scope

- Upload an image from any browser
- Store it as an object in a private S3 bucket
- List every stored image, newest first
- View the full image through a signed link
- Delete an image from the bucket
- Reject non-image files and anything over 10 MB

2 AWS services at the core

EC2 for compute, S3 for storage

0 files on the instance disk

the server stays disposable

11 nines of durability

S3 is built never to lose an object

BUILDING BLOCKS

The AWS services used

EC2

Elastic Compute Cloud

A t3.micro virtual machine running the Flask application on port 8080. Free Tier eligible.

S3

Simple Storage Service

Object storage holding every uploaded image under the uploads prefix of one private bucket.

IAM

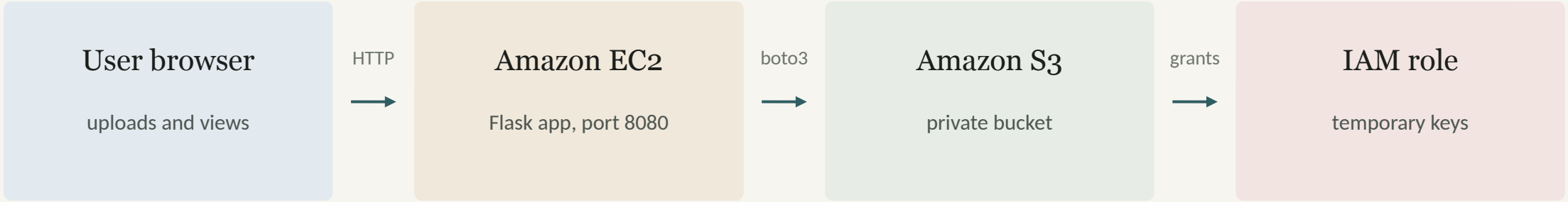
Identity and Access Management

A role on the instance that hands the application temporary keys, so no password sits in the code.

Deliberately small. No database, no load balancer, no Lambda: two services are enough to show the one idea that matters.

ARCHITECTURE

How the pieces fit together



EC2 stays stateless

Nothing is saved on the instance disk. Terminate it, launch a new one, and the gallery is unchanged.

S3 holds the truth

Objects live under uploads/ with a timestamped unique key, so two users can never overwrite each other.

No keys in the code

The role supplies rotating credentials, so nothing secret is ever committed to the repository.

The split between the two middle boxes is the whole project.

REQUEST FLOW

What happens when you press Upload

01 **Browser posts the file**

A multipart form sends the image to /upload on the instance.

02 **The app validates it**

The extension must be an image type and the body under 10 MB.

03 **A unique key is built**

uploads/20261010-141655-5e12be.png, a timestamp plus a random suffix.

04 **boto3 streams it to S3**

upload_fileobj sends the bytes through. Nothing touches the EC2 disk.

05 **The gallery re-reads S3**

list_objects_v2 returns every object, which the app sorts newest first.

06 **Signed links render it**

Each thumbnail gets a URL valid for one hour. The bucket stays private.

SECURITY

Keeping the bucket private

The shortcut, avoided

- Make the bucket public so image tags work
- Paste access keys into the source file
- Anyone guessing a URL reads every image
- A leaked key becomes a leaked account

What this project does

- Block Public Access stays switched on
- An IAM role gives rotating credentials
- Presigned URLs expire after one hour
- The role may only touch the uploads prefix

Least privilege in one line: the application may read, write and delete under uploads/, and nothing else in the account.*

IMPLEMENTATION

The part that talks to AWS


```
s3 = boto3.client("s3", region_name=REGION)

# upload: stream the file into the bucket
key = f"uploads/{stamp}-{uuid4().hex[:6]}{ext}"
s3.upload_fileobj(file, BUCKET, key,
        ExtraArgs={"ContentType": file.mimetype})

# list: every object, newest first
objects = s3.list_objects_v2(Bucket=BUCKET,
              Prefix="uploads/")["Contents"]

# view: a link that works for one hour
url = s3.generate_presigned_url("get_object",
        Params={"Bucket": BUCKET, "Key": key},
        ExpiresIn=3600)
```

boto3

The AWS SDK for Python. Each call is one HTTPS request to the S3 API.

No credentials

boto3 finds the instance role on its own. No access key in this file.

Unique keys

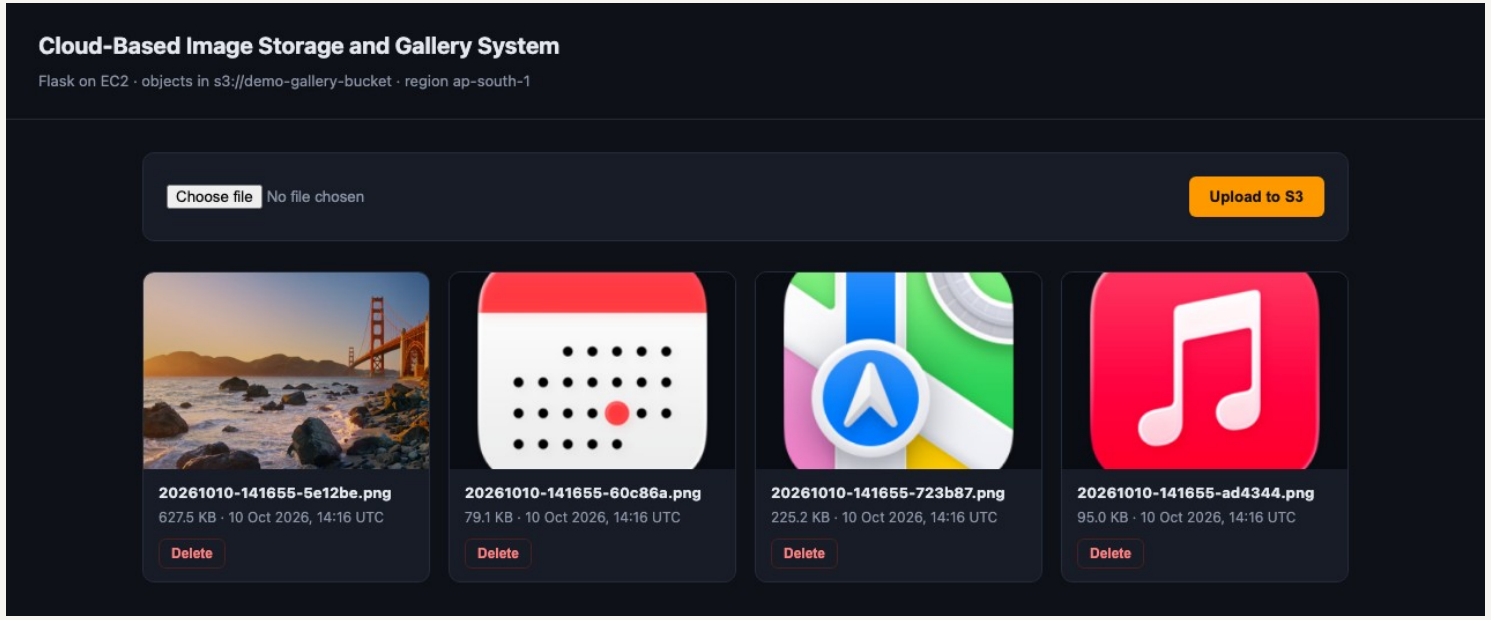
A timestamp plus six random characters, so uploads cannot collide.

Expiring links

A presigned URL is the bucket signing a time-limited permission.

DEMONSTRATION

The running application



The gallery with four objects in the bucket.

Upload

The panel posts the file straight to S3.

Cards

Each card is one object in the bucket.

Metadata

Size and time are read from S3 itself.

Thumbnails

Loaded over presigned URLs.

Delete

Removes the object, not a database row.

DEPLOYMENT

Getting it onto AWS

01

Create the bucket

Block Public Access stays switched on.

02

Create the IAM role

Read, write and delete on uploads/* only.

03

Launch t3.micro

Attach the role, open ports 22 and 8080.

04

Install and run

Set the bucket name, start the app with gunicorn.

05

Open the public IP

Upload, then confirm the object in the S3 console.

The whole setup takes about twenty minutes and stays inside the AWS Free Tier. Terminate the instance and empty the bucket afterwards to keep the bill at zero.

RESULTS

What was tested

TEST CASE	EXPECTED AND OBSERVED RESULT	STATUS
Upload a PNG	Object appears in the bucket, bytes identical to the file sent	**Pass**
View an image	Presigned URL returns HTTP 200 and the exact uploaded bytes	**Pass**
List the gallery	Every object listed newest first, with size and timestamp	**Pass**
Delete an image	Object removed, the gallery falls back to its empty state	**Pass**
Upload a .txt file	Rejected with a message, nothing written to the bucket	**Pass**
Upload with no file	Rejected with a message, no request sent to S3	**Pass**
Delete outside uploads/	Refused as an invalid key, no S3 call made at all	**Pass**

Every case was run against a live S3 API, including a byte-for-byte comparison of what each presigned URL returned.

SCOPE

Honest limits, and what comes next

Current limitations

- No login, so anyone with the address can upload
- One bucket holding one shared gallery
- Thumbnails are full-size downloads
- A single instance, so no failover

Where it goes next

Cognito logins

A private prefix per user in the same bucket.

Lambda thumbnails

Resize on upload through an S3 event trigger.

CloudFront

Cache images at the edge instead of signing each view.

DynamoDB metadata

Titles, tags and captions beside the objects.

Every extension plugs into the same two-service core, which is why the simple version was worth building first.

CONCLUSION

Separate the compute from the storage.

EC2 runs the code and can be thrown away at any moment. S3 keeps the images durable and independent of whichever server happens to be running. That single split is what makes the application scalable, and it is the whole lesson of this project.

Thank you. Questions welcome.

The one idea to remember